

Generative Adversarial Networks: From 2D Toy Distributions to Medical Images, Network Traffic, and Sketches

GAN Assignment Report

1. Introduction

This paper discusses a sequence of increasingly complex Generative Adversarial Network (GAN) experiments in two categories: (1) fully-connected GANs trained from scratch on synthetic 2D point distributions, designed to provide some intuition and to deliberately induce and solve the mode collapse; and (2) three Deep Convolutional GAN (DCGAN)-based models applied to real datasets from three very different domains — medical imaging (OCTMNIST retinal OCT scans), network security (CICIDS2017 intrusion detection traffic), and creative sketch generation (QuickDraw ‘birthday cake’ doodles). Rather than reciting the usual textbook knowledge about GANs, the focus here is on the choice of the specific architectures for each dataset, what the training dynamics and generated samples tell about the performance of the generator in learning the target distribution, and where the experiments are limited by computational power and data availability considerations.

All models use the same adversarial scheme: generator G and discriminator D trained via the non-saturating binary cross-entropy minimax loss function, Adam optimisation with the momentum parameter $\beta_1 = 0.5$ (which prevents oscillation between the generator and the discriminator, as recommended in the DCGAN paper [2]), and one-sided label smoothing (real samples get labels of 0.9 instead of 1.0 to prevent the discriminator from becoming overconfident and starving the generator of the gradient [7]). The only thing which changes between the experiments is the network architecture — simple MLPs for 2D points, transposed convolution stacks for images, and another MLP for high-dimensional tabular features — which reflects the geometric structure of each particular data type, see details in Section 2.

2. Methods

2.1 Fully-Connected GANs on 2D Distributions (Part 1)

The first part of the experiment reproduces the GAN tutorial's sine wave model and then deliberately shifts to the more difficult target distribution: a mixture of five isotropic Gaussians located in the plus-shape configuration. The sine wave and the noisy parametric curve are both continuous 1D manifolds, so the small network usually manages to learn them easily. In contrast, the five-component Gaussian mixture is a disconnected distribution consisting of five separate blobs with no connections between them, thus serving as a much better stress test — a small or poorly architected generator with the activations that create dead gradients will often fail to cover all five modes and stick to only one or two. That allows for a meaningful architecture comparison, as the exact same training procedure is applied twice to the same GMM distribution, once with the baseline generator/discriminator (3 layers, 64 hidden units, ReLU activations) and then with the modified generator (5 layers, 128 hidden units, Tanh activations in lieu of ReLU). Tanh is a bounded and bi-directionally smooth function, while ReLU may simply zero out and stop functioning (‘dying ReLU’) the units that could have been used to reach some outlier mode — this is the hypothesis checked here.

2.2 DCGAN Architecture for Image Data (Parts 2.1 and 2.3)

As both OCTMNIST (retinal OCT scans) and QuickDraw (sketches in the bitmap format) are 28×28 single-channel images, they both use the same DCGAN architecture, enlarged to 32×32 dimensions to allow for the division of the standard four-layer stack into equal $4 \rightarrow 8 \rightarrow 16 \rightarrow 32$ number of feature maps. The generator architecture follows the Radford et al. DCGAN scheme [2]: the vector $z \in \mathbb{R}^{100}$ is projected to the sequence of ConvTranspose2d layers ($256 \rightarrow 128 \rightarrow 64 \rightarrow 1$) each followed by the BatchNorm and ReLU activation (except the last layer which uses Tanh due to the image values being normalised to $[-1, 1]$). The discriminator mirrors this structure using the strided convolutions, BatchNorm and LeakyReLU(0.2) activation (LeakyReLU prevents the same problem as Tanh in the generator in the Part 1 due to the dead-units issue). Convolutions make sense here due to the spatial structure of the OCT scans and sketch images — pixels close to each other are correlated, and the objects of interest (retinal layers/fluid pockets in OCT, or cakes in sketches) consist of the specific edges and texture defined by their local properties which would require learning at each pixel separately by a fully-connected network. Also, each script allows for concatenating the one-hot encoded class vector to the generator input z and treating it as additional

channels in the discriminator input, which could allow for training the conditional GAN, though it is not used in the run (explained in Section 3).

2.3 MLP GAN for Tabular Network-Traffic Data (Part 2.2)

Finally, the CICIDS2017 traffic records are high-dimensional numeric vectors (78 features) consisting of the packet timing, byte counts, flow-duration statistics, etc. These features are not ordered in any way, neither spatially nor sequentially — there is nothing special about feature 12 compared to feature 70 or 13. Therefore, the convolutional network would create the false assumption about the locality of these features, so a plain multilayer perceptron (2 hidden layers of 256 units, ReLU activation in G and LeakyReLU(0.2) in D) is used instead, resembling the Part 1 architecture but scaled to the higher input dimensionality. Before the training, the features are z-scored using a StandardScaler for the generator to work equally well across all features, and the multi-class attack label is reduced to the binary signal indicating BENIGN vs. ATTACK classes due to their huge imbalance.

2.4 Evaluation Methodology

Two approaches to evaluation are utilized, both for images and tabulars, one qualitative and one quantitative. Qualitative evaluations of images come in the form of grids of generated images produced after certain training checkpoints. Quantitative evaluations use the same formula for the Fréchet distance as [3], but applied to feature statistics of the real and generated samples (mean and covariance matrix of the discriminator's penultimate-layer activations rather than of the Inception-network activations). While Inception-FID compares samples in the space of ImageNet-features, the target domain for the image generation tasks is far from natural RGB pictures, so raw Inception features are a poor metric space for these datasets. Besides, Inception weights require downloading from the internet, which is not guaranteed to succeed. The discriminator-features Fréchet distance preserves the same idea, namely whether the real and the fake samples lie in the same region of a learned feature space, with the feature space actually trained on the target domain but not numerically comparable between domains or to published Inception-FID values. As for tabulars, where a natural convolutional feature extractor does not exist, the real vs. generated samples can be compared using projections via PCA and t-SNE [8] plus interpretable statistics: difference between per-feature means/standard deviations and the mean nearest-real-neighbour distance of the synthetic samples. For Part 1, mode coverage is evaluated by simply assigning each generated point to its nearest of the five modes and counting the number of points near each mode, which is a much more straightforward and interpretable mode-collapse diagnostics compared to distributional distance in the case of a low-dimensional, precisely known target distribution.

3. Results and Analysis

3.1 Part 1: 2D Distributions — Fitting and Mode Collapse

The baseline architecture reproduces the sine wave distribution rather closely (Figure 1, left) after 4,000 training epochs, which confirms that the training loop and hyperparameters chosen work well before switching to a more challenging target. On the Gaussian mixture, the baseline architecture (Figure 1, centre) clearly fails to cover the distribution: generated points cluster around most of the five modes but fail to populate one region. The mode-coverage counts make this observation clear — of 1,000 generated points assigned to their nearest mode, the baseline covers 0 points near one of the five modes, while the other four receive between 202 and 265 points (Table 1). This is a textbook partial mode collapse: the generator discovered a locally stable way to fool the discriminator using only four of the five modes, and once the discriminator stopped punishing the lack of points in the fifth mode, there was no need to find a way to reach it via gradient descent.

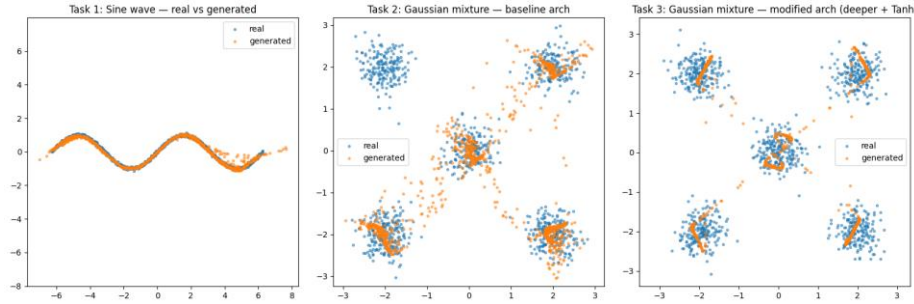


Figure 1. Real (blue) vs. generated (orange) points after training. Left: sine wave, baseline architecture. Centre: Gaussian mixture, baseline architecture — note the empty region near one mode. Right: Gaussian mixture, modified architecture (deeper, wider, Tanh) — all five modes are populated.

The modified architecture — five layers instead of three, 128 hidden units instead of 64, and Tanh instead of ReLU in the generator — covers all five modes with much more uniform counts (184–225 points each, Table 1). It supports the hypothesis stated in Section 2.1: extra depth/width gives the generator more capacity to implement multi-modal mapping, and the non-zero, smooth everywhere derivative of Tanh allows the very small part of latent space that needs to be mapped into the missing mode to get useful gradient during training instead of dead ReLU nodes.

Architecture	Mode 1	Mode 2	Mode 3	Mode 4	Mode 5	Modes with ≈ 0 coverage
Baseline (3 layers, ReLU)	240	202	0	265	221	1
Modified (5 layers, Tanh)	209	172	197	184	225	0

Table 1. Generated points (out of 1,000) assigned to each of the five Gaussian-mixture modes.

Loss curves (Figure 2) provide an interesting qualification to interpreting GAN training based on loss alone: both networks eventually converge to a similar range of generator/discriminator BCE loss ($G = 0.8$, $D = 1.25$ – 1.4) after 4,000 epochs, and the modified version's loss curves are not obviously 'better' to the eye — adversarial loss cannot reliably correlate with sample quality and mode coverage beyond initial training stabilization, precisely why an explicit mode-coverage count (Table 1) is a necessary complement rather than a redundant diagnostic to loss curves.

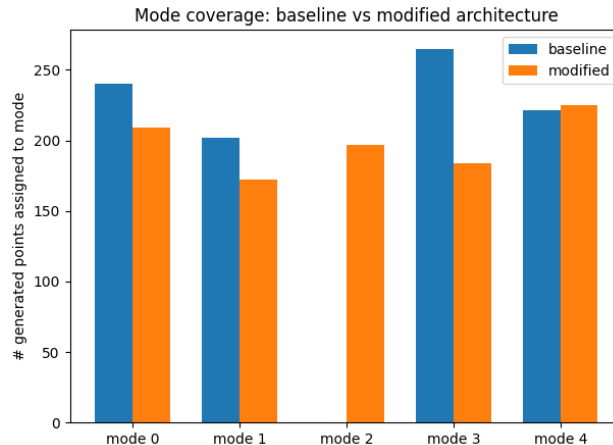


Figure 2 (right). Bar chart comparison of mode coverage between the baseline and modified GAN architectures for the five mixture components.

3.2 Part 2.1: OCTMNIST DCGAN

OCTMNIST dataset consists of 97,477 training images from four types of retinal image, heavily imbalanced towards NORMAL (47.2%) and choroidal neovascularization (CNV; 34.4%) classes, with drusen being the least common (8.0%, Figure 3). This run trains the unconditional DCGAN for the full 30-epoch training period, saving out generated sample grids at each fifth epoch (Figure 4) and plotting out the loss curve (Figure 5). As the checkpoint grids show, there is obvious qualitative improvement from epoch 5, where the generator already begins to produce the characteristic layered OCT structure, albeit with fuzzy boundaries; the layer boundaries become sharper and

localised bright/dark bands corresponding to retinal layers and pockets of fluid appear by epoch 15; by epoch 30, several samples display convincing, high-contrast layered OCT structure, comparable in style to real scans shown in Figure 3, although the rest remain malformed — characteristic of a generator that is making genuine progress but has yet to fully converge. The loss curve (Figure 5) presents an apparently paradoxical picture: discriminator loss continues to stay low throughout the run ($0.58 \rightarrow 0.37$) as generator loss increases ($2.5 \rightarrow 4.4$) and becomes noisier in the 30 epochs — the signature feature of a discriminator gaining increasing ground over the generator. Interpreted in isolation, this would seem to imply a generator that is unable to learn anything, whereas seen together with the sample checkpoint grids, it provides a concrete illustration of this report's primary methodological point (Section 4) about the inability of adversarial loss to serve as a stand-alone surrogate for sample quality — the discriminator's loss keeps getting lower even as the generator's samples become increasingly realistic, as BCE loss only represents the relative strengths of D and G and not the objective quality of the generator's samples. The final Fréchet distance between discriminator features is 1.02.

OCTMNIST — sample images per class

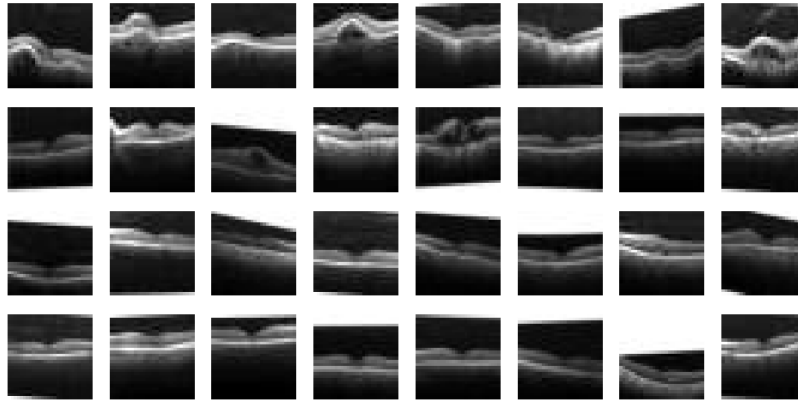


Figure 3. Sample OCTMNIST training images by class (CNV, DME, drusen, normal); 8 per class.

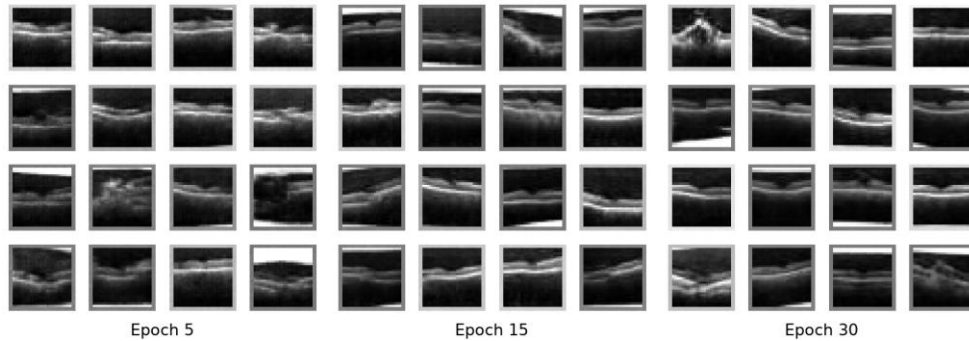


Figure 4. Generated-sample OCTMNIST DCGAN grids at epoch 5 (left), epoch 15 (centre), and epoch 30 (right), showing the gradual emergence of realistic OCT layers.

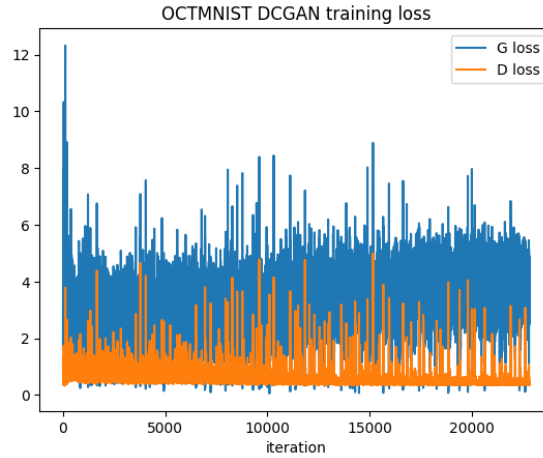


Figure 5. Training loss during 30 epochs of OCTMNIST DCGAN training process. In this case, D loss is consistently low while G loss increases and becomes more noisy; it is tempting to read this as the generator being underperforming, but looking at Figure 4 one can see significant qualitative improvement during these 30 epochs (Section 4).

3.3 Part 2.2: CICIDS2017 Tabular GAN

The CICIDS2017 dataset collected on Wednesday contains 557,953 records with 78 features, including 312,262 BENIGN flows, 231,073 DoS-Hulk flows and much smaller amounts of the other three DoS types: slowloris, Slowhttptest and GoldenEye; that is why the task reduces labels to just two binary options: BENIGN or ATTACK, and does not attempt to condition the generation on five highly unbalanced classes. As we can see from Figure 6 below, D loss decreases (0.81→0.56) and G loss increases (1.90→3.16) in later epochs; this once again indicates increasing advantage of the discriminator over generator, what is a fairly typical pattern for tabular GANs. The explanation for this pattern is in structured nature of numeric features in a tabular dataset; many of those are counts, rates or durations, with clear boundaries of their possible values in the real world; therefore, it is easier for the discriminator to distinguish them from the generator's unconstrained continuous output.

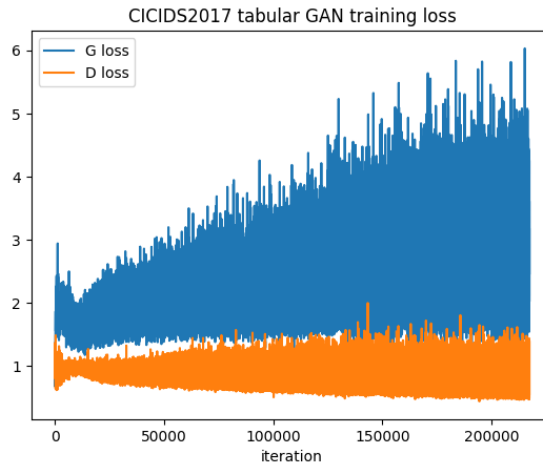


Figure 6. CICIDS2017 GAN training loss over 100 epochs.

PCA projection (Figure 7, left) clearly shows that real and synthetic points occupy very similar region in the space of principal components, but the distribution of synthetic points is noticeably denser and lacks some outliers, present in the real dataset. The latter means that generator has learned the major distribution modes well, but not the tail; the t-SNE projection, which is much more sensitive to local neighbourhood, makes this even more evident: several small clusters of real points (possibly representing different attack subtypes or protocol/port combination) lack synthetic points in the projection space, thus generator is smoothing the structure of the data, not captured by PCA projection, which is linear and not sensitive to local features. The latter idea is confirmed by alignment metrics: mean |real-fake| difference between the distributions' feature means is 0.177 standard-score units, mean |real-fake| difference between standard deviations is larger 0.506, and the mean distance between a synthetic point and its closest real neighbour is 1.159 – the ratio of the standard-deviation to mean-difference of the latter pair (roughly 3×) is the quantitative

confirmation that generator learns the central tendency better than the dispersion of the distribution, i.e. generator generates 'safe'/average traffic.

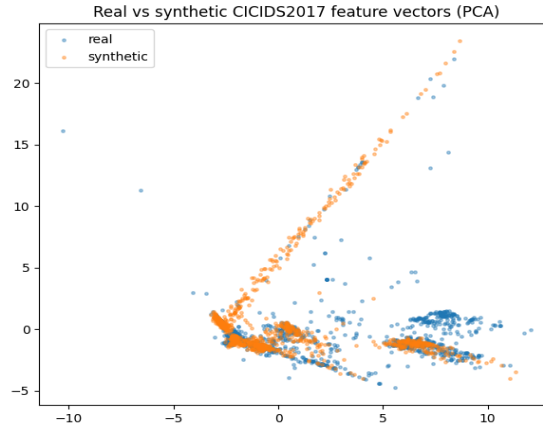


Figure 7a. PCA projection of real vs. synthetic CICIDS2017 feature vectors.

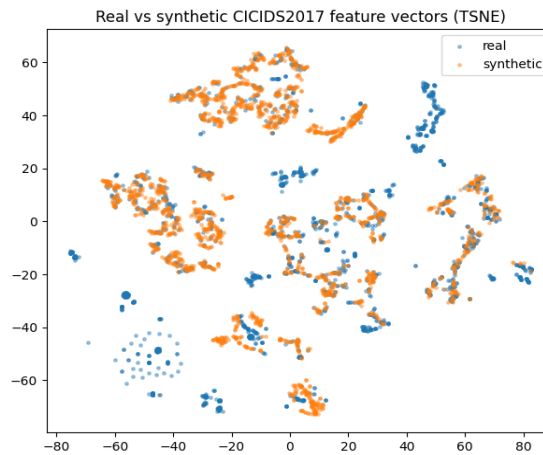


Fig. 7b. t-SNE of the same real/synthetic dataset pair, highlighting the generator's failure to capture some local structure.

3.4 Part 2.3: QuickDraw 'Birthday Cake' DCGAN

20,000 sketches from the QuickDraw 'birthday cake' class (Fig. 8) were used as input and the same DCGAN model architecture as was applied to OCTMNIST was trained for 25 epochs (Fig. 9). The output sample grids at epochs 5, 15, and 25 (Fig. 9) demonstrate the expected qualitative improvement: while the outputs at epoch 5 are featureless grey blobs with only hints of a vertical/tiered structure, at epoch 15 the cake-like silhouettes with candle-like projections become apparent, although they are still very noisy; at epoch 25, several high-contrast tiered-cakes emerge in close approximation to the style of the actual bitmap sketches, although others are still malformed – an expected situation for a GAN that is making real progress in training but has not fully converged. The training loss graph (Fig. 10) demonstrates considerably more noise than the graphs for CICIDS and Part-1, including a sharp spike in discriminator loss (to 1.50) at epoch 13, accompanied by a temporary collapse in generator loss (to 0.11) before it recovers by epoch 15 – a helpful reminder that transient instability during adversarial training does not necessarily compromise the eventual result. The final Fréchet distance between the real and generated distributions of discriminator features is 0.57, lower than OCTMNIST's 1.02 – consistent with, if not a proof of, the above-mentioned impression that the birthday cake generator (25 epochs, simpler single-object sketches) achieved a better qualitative match to its real-data distribution than the OCTMNIST generator (30 epochs, more complex multi-class medical imagery with four visually similar sub-classes) achieved to its own. Given that OCTMNIST actually had more training epochs, the remaining difference seems more likely to be due to data complexity (four imbalanced anatomical classes vs. one visually distinct category of sketches) than to a difference in training budget, anyway – and the two Fréchet distances can hardly be compared quantitatively because they are not even measured on the same feature scale. The final extension task – training a GAN for the additional 'cat', 'house' QuickDraw categories – was impossible to execute due to the absence of the necessary .npy bitmap files in the local data directory. This is a data limitation issue rather than a method limitation issue, and is the only part of Part 2.3 that is unfinished, since the birthday cake training, evaluation and figures are complete.

QuickDraw sample: birthday cake

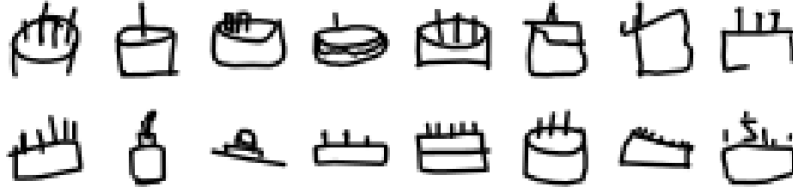


Fig. 8. Sample QuickDraw 'birthday cake' sketches (16 of 20,000 training images).



Fig. 9. Outputs at epoch 5 (left), epoch 15 (centre), and epoch 25 (right), demonstrating progressive development of the tiered cake structure.

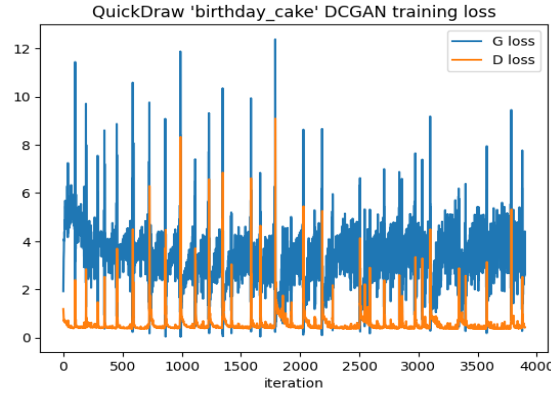


Fig. 10. Training loss for 25 epochs of the QuickDraw DCGAN training, including a transient instability spike at epoch 13.

4. Discussion

When considered together, the lessons from the four experiments are essentially the same: loss curves of a GAN model are insufficient as an informative metric to determine the quality of generated samples; rather, depending on the type of data, a suitable complementary diagnostic metric should be used. As shown in Part 1, where the true distribution was precisely known, a direct mode-coverage count served as the most informative diagnostic metric and distinguished between the two models despite their similar final loss values. For the image data, a combination of domain-specific Fréchet distance and periodic checkpoint grids makes it abundantly clear that, without such a complementary metric, loss curves would actively mislead: in case of OCTMNIST, the loss of the discriminator was consistently low, while that of the generator was increasing and became noisy throughout the 30 epoch-long process – which can be read as 'the generator is losing' – while the grids of checkpoints (Figure 4) demonstrate its steady generation of increasingly realistic layered retinal structure at the same time. QuickDraw's loss curve is noisy even more and includes a training instability spike in the middle, yet the grids of checkpoints (Figure 9) demonstrate an equally clear progress towards realistic cakes. These examples provide the clearest illustration for the claim made above: checkpoint grids, not the loss curve, are the metric which tracks the generator learning process. In the tabular experiment, where there was no natural CNN feature space to apply the Fréchet distance, PCA/t-SNE and moment-matching served the same diagnostic purpose and helped reveal a particular failure mode (under-matched variance / missed local clusters) that could be hidden by a single scalar score.

The architecture choices make the same general point, namely: align the inductive bias of the network to the structure of the data, not stick to the same architecture in all cases. Convolutions were the appropriate choice for OCTMNIST and QuickDraw as both datasets involve spatially local image data; the same convolutional network would be an arbitrary and wrong choice for the unordered tabular features of CICIDS2017, where a plain MLP does not assume

the false presence of locality. Similarly, changing ReLU for Tanh in Part 1's generator was a small and targeted tweak motivated by a specific failure mode (dead units starving an isolated mode of gradient) rather than a generic 'bigger is better' upgrade, and the mode-coverage results back the claim that it was the right lever to address this pathology.

5. Conclusion

In the course of four experiments covering synthetic 2D points, retinal OCT images, network-intrusion tabular data, and hand-drawn sketches, alignment of generator/discriminator architecture to the geometry of the data (MLP models for unordered points/features, convolution for spatially local images), coupled with domain-specific evaluation (exact mode counts, domain-specific Fréchet distance, or projection-based distributional comparison) turned out more informative than reading the loss curves. All four core experiments have been executed to completion with both quantitative and qualitative evaluation performed: architecture comparison in Part 1, the OCTMNIST DCGAN (30 epochs, full checkpoint grids), the CICIDS2017 tabular GAN, and the QuickDraw DCGAN (25 epochs). The most clear-cut positive result is the architecture comparison in Part 1, which demonstrated the ability of a small and motivated tweak (depth, width, Tanh) to eliminate mode collapse. The clearest area for further work is not the training budget, which is sufficiently large in both image experiments now, but the conditional-GAN extensions: exercising the class-conditioning already implemented in the shared code would check whether it helps to improve per-class fidelity, especially in OCTMNIST's imbalanced classes.

References

- [1] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative Adversarial Networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.
- [2] A. Radford, L. Metz, and S. Chintala, "Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks," in *International Conference on Learning Representations (ICLR)*, 2016.
- [3] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [4] J. Yang, R. Shi, D. Wei, Z. Liu, L. Zhao, B. Ke, H. Pfister, and B. Ni, "MedMNIST v2 — A Large-Scale Lightweight Benchmark for 2D and 3D Biomedical Image Classification," *Scientific Data*, vol. 10, no. 41, 2023.
- [5] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, 2018.
- [6] J. Jongejan, H. Rowley, T. Kawashima, J. Kim, and N. Fox-Gieg, "The Quick, Draw! Dataset," Google Creative Lab, 2016. [Online]. Available: <https://github.com/googlecreativelab/quickdraw-dataset>
- [7] T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, and X. Chen, "Improved Techniques for Training GANs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.
- [8] L. van der Maaten and G. Hinton, "Visualizing Data using t-SNE," *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008.

GITHUB LINK: <https://github.com/Sadweep03/Generative-modelling-case-study>